01 El proyecto, de principio a fin

Una guía para entender qué hace cada parte y explicar el código que la conecta.

QUÉ ES

¿Hay cupo? es una plataforma web para buscar u ofrecer arriendos compartidos y cupos de vivienda. Su núcleo reúne cuentas de usuario, publicaciones, filtros y perfiles.

Problema, usuarios y objetivo

Problema: comparar avisos requiere revisar ubicación, precio, cupos y condiciones. La propuesta reúne esa información en un solo lugar.
Usuarios: personas mayores de edad que buscan vivienda o roomies, y personas que ofrecen un espacio disponible.

Objetivo general: facilitar la búsqueda y oferta de arriendos compartidos mediante una plataforma que centraliza publicaciones y datos de usuarios.

Objetivo específico	Dónde se ve en el proyecto
Gestionar cuentas y perfiles	Registro, login y pantalla de perfil.
Administrar avisos	Crear, consultar, editar y eliminar publicaciones.
Facilitar la búsqueda	Filtros por ubicación, cupos, tipo de vivienda y mascotas.
Integrar interfaz y datos	JavaScript → FastAPI → PostgreSQL → respuesta JSON.

El recorrido principal

Crear cuenta / ingresar → explorar avisos → revisar detalles → publicar o administrar avisos → editar el perfil. El backend también contiene solicitudes de conexión y matches; esa parte aún no tiene una pantalla integrada.

Cómo leer esta guía

Páginas	Qué aprenderás
2 - 3	Cómo arranca la aplicación y cómo funciona el acceso.
4 - 5	Cómo viaja un formulario, se valida y se guarda; CRUD.
6 - 8	Listado y filtros, perfil, servicios externos y conexiones.
9	Guion para exponer, conceptos y preguntas de defensa.

Sobre los extractos: se copian del código real, con archivo y numeración original. Las omisiones están marcadas; las líneas largas se ajustan visualmente. Los objetivos se formularon a partir de las funciones del repositorio.

02 Estructura y arranque

El archivo principal une las pantallas, las rutas de la API y los módulos de negocio.

A. Crear la aplicación e incorporar los módulos

main/main.py | L54, L65-66, L71-73

```
54  app = FastAPI(lifespan=lifespan)
    [líneas intermedias omitidas]
65  app.include_router(conexion_router)
66  app.include_router(auth_router)
    [líneas intermedias omitidas]
71  app.mount("/imagenes", StaticFiles(directory=IMAGENES), name="imagenes")
72  app.mount("/Iconos", StaticFiles(directory="Iconos"), name="iconos")
73  app.mount("/PerfilU", StaticFiles(directory="PerfilU"), name="perfil_u")
```

FastAPI organiza las rutas. **include_router** agrega autenticación y conexiones. **mount** publica archivos estáticos, como imágenes y recursos del perfil. En este arranque los módulos comparten una aplicación.

B. Una ruta puede devolver una pantalla HTML

main/main.py | L84-90

```
84  @app.get("/")
85  def mostrar_login():
86      return FileResponse(FRONTEND_PRINCIPAL / "Auth_app(login)" / "auth_app.html")
87
88  @app.get("/inicio")
89  def mostrar_pagina_principal():
90      return FileResponse(FRONTEND_PRINCIPAL / "hayCupo.html")
```

GET / entrega el login, **GET /inicio** entrega la página principal y **GET /crear** el formulario. **FileResponse** devuelve un archivo; las rutas **/api/...** devuelven datos para JavaScript.

Mapa de carpetas: qué debes reconocer

Carpeta / archivo	Responsabilidad
main/main.py	Servidor principal; publicaciones, perfil y ubicaciones.
auth_service/	Registro, validación de credenciales y cookie de acceso.
match_conexion_service/	Modelos y rutas de solicitudes, matches y verificación.
PaginaPrincipal/	Inicio, autenticación y “Mis publicaciones”.
Creacion_De_publicacion/ · PerfilU/	Formulario de avisos y pantalla de perfil.
tests/test_match_conexiones.py	Pruebas automatizadas del módulo de conexiones.

Para decir: “La interfaz usa HTML, CSS y JavaScript. JavaScript pide datos a FastAPI; Python consulta PostgreSQL y responde al navegador. Uvicorn ejecuta el servidor”.

03 Registro, login y sesión

El navegador envía las credenciales; el servidor consulta la cuenta y responde.

A. Registro: datos y validaciones

RegistroRequest recibe nombre, correo, contraseña y fecha de nacimiento. **BaseModel** describe la solicitud; **EmailStr** valida el correo. Se exige tener 18 años, contraseña de al menos 6 caracteres y correo no repetido (auth_service/main.py, L30-68).

B. El navegador hace una petición POST con JSON

PaginaPrincipal/Auth_app(login)/auth_app.js | L86-92

```
86  const respuesta = await fetch('/auth/login', {
87      method: 'POST',
88      headers: { 'Content-Type': 'application/json' },
89      body: JSON.stringify(payload)
90  });
91
92  const json = await respuesta.json();
```

payload contiene correo y contraseña. **JSON.stringify** lo convierte a texto; **await** espera la respuesta; **respuesta.json()** recupera el objeto devuelto por el servidor.

C. El servidor busca la cuenta y entrega una cookie

auth_service/main.py | L106-115

```
106 usuario = conexion.execute(
107     "SELECT id_usuario, nombre FROM haycupo.usuario WHERE correo = %s AND contrasena = %s",
108     (datos.correo, datos.contrasena)
109 ).fetchone()
110
111 if not usuario:
112     raise HTTPException(status_code=401, detail="Correo o contraseña incorrectos.")
113
114 # Guardar la cookie de sesión en el navegador
115 response.set_cookie(key="roomies_id", value=str(usuario[0]), httponly=True, path="/")
```

%s + valores separados hacen una consulta parametrizada. **fetchone()** obtiene una fila; si no existe, devuelve error **401**. Si coincide, se establece **roomies_id** con el ID del usuario.

D. Publicaciones y perfil leen ese ID

main/main.py | L77-78

```
77  def autor_de_prueba(roomies_id: str = Cookie(default="1")):
78      return int(roomies_id)
```

LÍMITE IMPORTANTE DEL PROTOTIPO

La cookie guarda el ID directamente y, si falta, el código usa el usuario 1. Las contraseñas se comparan sin hash. El texto **demo_cookie_token** devuelto al cliente no es un JWT. Cerrar sesión desde el inicio limpia localStorage, pero no llama a la ruta que elimina la cookie.

04 Crear o editar una publicación

Sigue el dato desde el formulario: preparar → enviar → validar → guardar.

A. El formulario adapta los campos y decide POST o PUT

Creacion_De_publicacion/publicacion.js | L44-54, L60-67

```
44  const formDataOriginal = new FormData(formulario);
45  const formDataEnvio = new FormData();
46
47  formDataEnvio.append("titulo", formDataOriginal.get("titulo"));
48  formDataEnvio.append("categoria", formDataOriginal.get("categoria"));
49  formDataEnvio.append("ubicacion", formDataOriginal.get("ubicacion"));
50  formDataEnvio.append("precio", formDataOriginal.get("precio"));
51  formDataEnvio.append("tipo_vivienda", formDataOriginal.get("tipo"));
52  formDataEnvio.append("descripcion", formDataOriginal.get("contenido"));
53  formDataEnvio.append("tipo", "Compartida");
54  formDataEnvio.append("cupos_totales", formDataOriginal.get("cupos"));
    [líneas intermedias omitidas]
60  const url = idEdicion ? `/api/publicaciones/${idEdicion}` : "/api/publicaciones";
61  const metodo = idEdicion ? "PUT" : "POST";
62
63  try {
64      const respuesta = await fetch(url, {
65          method: metodo,
66          body: formDataEnvio
67      });
```

FormData reúne los campos. **append** adapta nombres: “tipo” pasa a **tipo_vivienda** y “contenido” a **descripcion**. Si hay ID de edición, usa **PUT** y una URL con ID; si no, **POST**.

B. FastAPI define qué valores acepta al crear

main/main.py | L199-208, L211-213

```
199 @app.post("/api/publicaciones", status_code=201)
200 def crear_publicacion(
201     titulo: Annotated[str, Form(min_length=1, max_length=200)],
202     categoria: Annotated[Literal["busco", "ofrezco"], Form()],
203     ubicacion: Annotated[str, Form(min_length=1, max_length=250)],
204     precio: Annotated[Decimal, Form(ge=0, max_digits=14, decimal_places=2)],
205     tipo: Annotated[Literal["Arrendador", "Compartida"], Form()],
206     cupos_totales: Annotated[int, Form(ge=1, le=2147483647)],
207     tipo_vivienda: Annotated[Literal["casa", "departamento", "habitacion"], Form()],
208     descripcion: Annotated[str, Form(min_length=1, max_length=5000)],
    [líneas intermedias omitidas]
211     imagen: Annotated[UploadFile | None, File()] = None,
212     id_usuario: int = Depends(autor_de_prueba)
213 ):
```

Literal restringe opciones; **ge=0** impide precios negativos; **ge=1** exige al menos un cupo; **Form** lee campos del formulario; **File** admite un archivo; **Depends** obtiene el autor.

ENTRADA Y SALIDA DEL CASO DE USO

Entra: título, categoría, ubicación, precio, cupos, vivienda y descripción. **Sale:** error **422** si los datos no cumplen; respuesta **201** con ID cuando la creación completa tiene éxito. La imagen se acepta en la API, pero el JavaScript actual del formulario no la añade al envío.

05 Base de datos y CRUD

La información no queda solo en pantalla: se inserta y actualiza en PostgreSQL.

A. El INSERT guarda el aviso y recupera su ID

main/main.py | L243-255

```
243 fila = conexion.execute(
244     """
245     INSERT INTO haycupo.publicacion (
246         id_usuario, tipo, titulo, descripcion, precio, ubicacion,
247         estado, cupos_totales, categoria, tipo_vivienda,
248         permitir_comentarios, mascota, contenido
249     )
250     VALUES (%s, %s, %s, %s, %s, %s, 'Disponible', %s, %s, %s, %s, %s, %s)
251     RETURNING id_publicacion
252     """,
253     (id_usuario, tipo, titulo, texto_desc, precio, ubicacion,
254      cupos_totales, categoria, tipo_vivienda, permitir, mascota, texto_desc),
255 ).fetchone()
```

psycopg ejecuta la consulta. **haycupo** es el esquema; **publicacion** es la tabla. Los **%s** reciben los valores de la tupla, sin concatenarlos al SQL. **RETURNING** obtiene el ID para relacionar una imagen y responder al cliente.

B. El UPDATE busca la publicación y su autor

main/main.py | L292-298

```
292     conexion.execute("""
293         UPDATE haycupo.publicacion
294         SET titulo=%s, categoria=%s, ubicacion=%s, precio=%s,
295             tipo_vivienda=%s, descripcion=%s, cupos_totales=%s, mascota=%s, contenido=%s
296         WHERE id_publicacion=%s AND id_usuario=%s
297     """, (titulo, categoria, ubicacion, precio, tipo_vivienda, descripcion, cupos_totales, mascota,
        descripcion, id_pub, id_usuario))
298 return {"mensaje": "Publicación actualizada correctamente"}
```

WHERE id_publicacion ... AND id_usuario ... restringe las filas a actualizar. Esta condición depende del ID de la cookie de prototipo. El mensaje de éxito no comprueba cuántas filas fueron modificadas.

Operación	Método y ruta	Qué hace
Crear	POST /api/publicaciones	Inserta un aviso.
Consultar	GET /api/publicaciones/{id}	Obtiene un aviso por ID.
Actualizar	PUT /api/publicaciones/{id}	Guarda los cambios.
Eliminar	DELETE /api/publicaciones/{id}	Borra el aviso y registros de imagen.

Eliminar desde la pantalla: MisPublicaciones.js, L63-65, envía fetch a la ruta del aviso con `method: 'DELETE'`. Tras confirmar, se recarga la lista. **CRUD** significa crear, consultar, actualizar y eliminar. La autorización del borrado requiere revisión.

06 Listado, tarjetas y filtros

Los avisos llegan desde la API; los filtros visibles trabajan en JavaScript.

A. Descargar los avisos y dibujar las tarjetas

PaginaPrincipal/HayCupo.js | L127-129

```
127  const respuesta = await fetch("/api/publicaciones");
128  window.todasLasPublicaciones = await respuesta.json();
129  renderizarPublicaciones(window.todasLasPublicaciones);
```

La API consulta publicaciones e imágenes con un **LEFT JOIN** y devuelve una lista JSON. **todasLasPublicaciones** conserva la lista; **renderizarPublicaciones** genera las tarjetas. Cada tarjeta muestra título, precio en CLP, ubicación y cupos.

B. Conservar solo los avisos que cumplen los filtros

PaginaPrincipal/HayCupo.js | L144-160

```
144  const filtradas = window.todasLasPublicaciones.filter(pub => {
145      const pubUbi = (pub.ubicacion || "").toLowerCase();
146      const coincideUbicacion = txtUbicacion === "" || pubUbi.includes(txtUbicacion);
147
148      // Usando cupos_totales
149      const pubPersonas = String(pub.cupos_totales || "").trim();
150      const coincidePersonas = valPersonas === "" || (valPersonas === "3" ? Number(pubPersonas) >= 3 :
         pubPersonas === valPersonas);
151
152      // Usando tipo_vivienda
153      const pubTipo = String(pub.tipo_vivienda || "").toLowerCase().trim();
154      const coincideTipo = valTipo === "" || pubTipo === valTipo;
155
156      const pubMascotas = String(pub.mascota || "").toLowerCase().trim();
157      const coincideMascotas = valMascotas === "" || pubMascotas === valMascotas;
158
159      return coincideUbicacion && coincidePersonas && coincideTipo && coincideMascotas;
160  });
```

Cómo leer la condición

Expresión	Significado
toLowerCase() / trim()	Normaliza minúsculas y espacios para comparar.
valor === "" || coincide	Si el filtro está vacío, acepta cualquier valor.
valPersonas === "3"	La opción 3 acepta avisos con tres o más cupos.
A && B && C && D	El aviso debe cumplir los cuatro criterios.

EJEMPLO FÁCIL PARA EXPLICAR

Si selecciono “departamento”, dos personas y mascotas “sí”, quedan los avisos que cumplen esas tres condiciones. La ubicación vacía no restringe el resultado. Después se redibuja la lista filtrada (L162).

Para la demo: el detalle usa el índice de la lista original (L168-169); tras filtrar puede abrir otro aviso. Muestra primero el detalle y luego los filtros. Es un pendiente de integración de la vista.

07 Perfil, imágenes y ubicaciones

El perfil muestra datos persistidos; los recursos externos amplían la experiencia.

A. Python convierte la fila del usuario en una respuesta

main/main.py | L326-335

```python
return {
    "nombre": user[0] or "Usuario",
    "edad": user[1] or "",
    "ocupacion": user[2] or "",
    "ciudad": user[3] or "",
    "breve_descripcion": user[4] or "Aún no hay una descripción breve.",
    "sobre_mi": user[5] or "No hay detalles adicionales.",
    "foto_perfil": user[6] or None,
    "total_publicaciones": pubs
}
```

Antes de responder, consulta al usuario y cuenta sus publicaciones. Las claves del JSON indican a JavaScript qué mostrar. **user[0]**, **user[1]**, etc. son columnas de la fila; **or** define un valor de respaldo.

B. Guardar el perfil y volver a cargar la vista

PerfilU/perfil.js | L87-97

```javascript
const formData = new FormData(e.target);

try {
    const respuesta = await fetch("/api/perfil", {
        method: "PUT",
        body: formData
    });

    if (respuesta.ok) {
        document.getElementById("modal-editar").style.display = "none";
        cargarPerfil(); // Recarga los datos actualizados sin refrescar la página
```

FormData(e.target) reúne datos y archivo del formulario. **PUT /api/perfil** actualiza edad, ocupación, ciudad, descripciones y foto; **cargarPerfil()** consulta de nuevo lo guardado. El endpoint no edita nombre ni correo.

Recurso	Implementación y estado
Fotos de publicaciones	La creación en API limita a 5 MB y verifica JPG, PNG o WEBP con Pillow. Falta enviar el archivo desde el formulario de avisos.
Foto de perfil	El formulario sí envía FormData. El backend guarda el archivo en main/imagenes; su validación es más limitada que la del nuevo aviso.
Ubicaciones	GET /api/ubicaciones consulta Geoapify y requiere clave configurada. Aún no está consumido por el formulario actual.

Consulta externa: en main/main.py, L126-129, **HTTPX** consulta Geoapify con un máximo de 10 segundos. **raise_for_status()** detecta errores HTTP; la API devuelve textos de ubicación. La clave se obtiene de configuración.

08 Conexiones, match y pruebas

Un match se registra cuando una solicitud de conexión es aceptada.

A. Los estados de una solicitud

match_conexion_service/models.py | L9-12

```python
class EstadoConexion(enum.Enum):
    Pendiente = "Pendiente"
    Aceptada = "Aceptada"
    Rechazada = "Rechazada"
```

Una **conexión** relaciona al solicitante con el receptor. El envío bloquea solicitudes a uno mismo y duplicadas pendientes o aceptadas. Aceptar o rechazar cambia su estado; la verificación es una marca adicional.

B. Al aceptar, se crea el registro de match

match_conexion_service/routes.py | L61-69

```python
if conexion_db.estado != EstadoConexion.Pendiente:
    raise HTTPException(status_code=400, detail=f"Esta solicitud ya fue {conexion_db.estado.value}.")

conexion_db.estado = EstadoConexion(respuesta.estado.value)
if respuesta.estado.value == EstadoConexion.Aceptada.value:
    db.add(Match(id_conexion_FK=id_conexion))
db.commit()
db.refresh(conexion_db)
return conexion_db
```

Este módulo usa **SQLAlchemy** para trabajar con objetos y tablas. **Entra:** el ID y el estado solicitado. **Hace:** exige que la solicitud siga pendiente y actualiza el estado. **db.add** prepara el match; **db.commit** confirma los cambios; **db.refresh** recarga la conexión guardada.

Relaciones: usuario → publicaciones → imágenes. Solicitante + receptor → conexión → aceptación → match. Aquí “match” significa conexión aceptada; el código no calcula un porcentaje de compatibilidad.

C. Una prueba comprueba que aceptar genera el match

tests/test_match_conexiones.py | L76-82

```python
aceptada = responder_solicitud(
    solicitud.id_conexion,
    ConexionUpdate(estado=EstadoConexion.Aceptada),
    self.db,
)
self.assertEqual(aceptada.estado, EstadoConexion.Aceptada)
self.assertEqual(self.db.query(Match).count(), 1)
```

assertEqual comprueba el resultado esperado: estado aceptado y un registro en la tabla de matches. El conjunto también prueba filtros, autosolicitudes, duplicados y verificación.

RESULTADO COMPROBADO

3 pruebas aprobadas con SQLite en una base temporal en memoria. Validan lógica del módulo de conexiones; no prueban el navegador, la autenticación ni la integración con PostgreSQL remoto.

09 Qué decir en la presentación

Usa los fragmentos para explicar el recorrido de los datos, sin memorizar funciones completas.

Tiempo	Qué mostrar y decir
0:00 - 0:45	**Problema y objetivo.** “Reunimos avisos y perfiles para facilitar la búsqueda y oferta de vivienda compartida”.
0:45 - 1:30	**Estructura.** Navegador → FastAPI → PostgreSQL. Muestra los routers de la página 2.
1:30 - 3:00	**Flujo principal.** Login, inicio y formulario. Explica FormData, validación e INSERT (páginas 3-5).
3:00 - 4:00	**Otras funciones.** Filtros, perfil y regla que convierte una aceptación en match.
4:00 - 5:00	**Resultados.** CRUD implementado, pruebas de conexiones y siguientes mejoras.

Preguntas rápidas que debes dominar

¿Qué es una API? Las rutas que permiten a la interfaz pedir o guardar datos.
¿GET, POST, PUT y DELETE? Consultar, crear, actualizar y eliminar.
¿JSON y FormData? JSON transporta objetos como texto; FormData transporta campos y puede incluir archivos.
¿Validar solo en el formulario basta? No. La API debe revisar las solicitudes aunque no provengan de la pantalla.
¿SQL parametrizado? La instrucción y sus valores viajan separados; los %s no se rellenan concatenando texto.
¿Por qué guardar en una base? Para consultar y mantener los datos después de cerrar o recargar el navegador.

Estado real y próximos pasos

1. Sesiones y permisos: usar contraseñas con hash, sesiones verificables, cierre de sesión completo y autorización por usuario; retirar las credenciales incrustadas del código.
2. Integración de datos: unificar los modelos del módulo de conexiones con los de publicaciones; hoy difieren en esquema, IDs y algunos campos.
3. Funciones pendientes: conectar la pantalla de conexiones, imagen del aviso y sugerencias de ubicación; chat y favoritos solo aparecen en el menú.
4. QA: corregir el detalle tras filtrar, reforzar validaciones de edición/borrado y probar los recorridos completos.

Arranque para una demo preparada

DESDE LA RAÍZ DEL REPOSITORIO

```
uvicorn main.main:app --reload
```

Luego abrir **http://127.0.0.1:8000**. Requiere las dependencias, acceso a la base configurada y sus tablas. Antes de exponer, ensaya el recorrido completo con datos de prueba.

Cierre: “El proyecto implementa la base de una plataforma de arriendos compartidos. Podemos explicar cómo cada dato viaja desde la interfaz hasta la base, y tenemos identificadas las integraciones y mejoras que faltan”.

Fuentes y alcance: los encabezados de cada extracto indican archivo y líneas. Arranque: documentacion_requisitos_auth.md, L13. Las guías manuales son planificación de QA. Revisión al 04/11/2026: lectura del código y ejecución de las tres pruebas citadas; sin ejecutar la aplicación ni consultar la base remota.